기초 Unity 스터디 3강

프리팹과 동적 생성

1. 다양한 오브젝트 추가하기

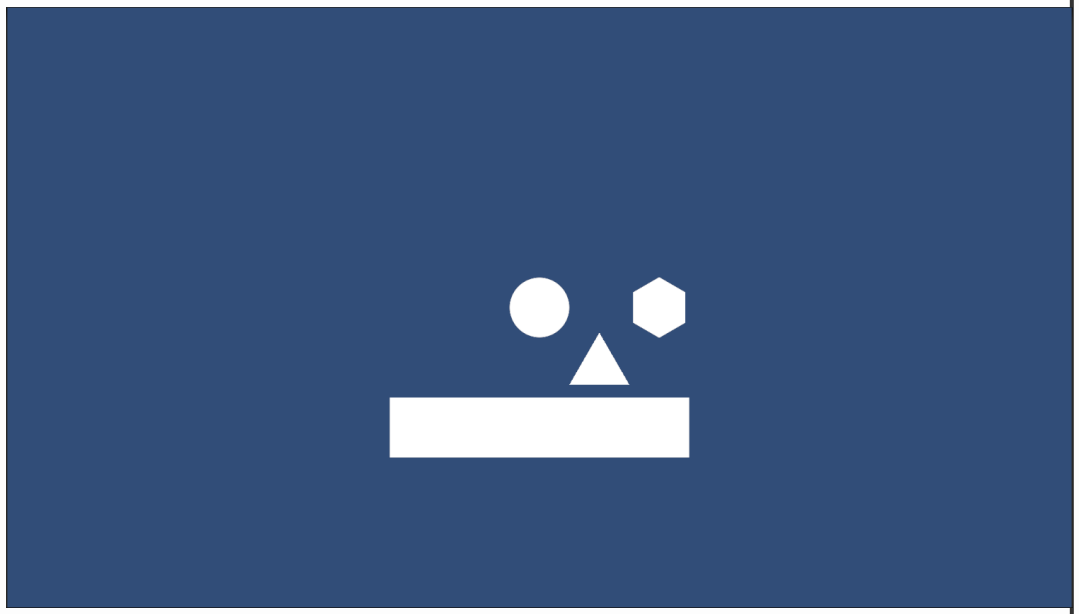
지난 시간 복습

- 플레이어와 플랫폼 오브젝트 생성
- 플레이어는 움직일 수 있고, 플랫폼은 딛고 서 있을 수 있음

- 이것만으로 플랫포머 게임이라고 하기는 아쉽다
  - 장애물: 닿으면 게임 오버
  - 포탈: 닿으면 다음 스테이지로 넘어감

가시와 포탈 구현하기

- 둘 다 오브젝트를 생성해 보자
  - 장애물
    - Sprite-Triangle
    - 이름: Spike
  - 포탈
    - Sprite-Hexagon Point Top
    - 이름: Portal
- Rigidbody2D와 Box Collider 2D도 추가
  - Rigidbody2D에서 Body Type은 Kinematic 또는 Static으로 설정
  - 콜라이더는 추가하면 저절로 스프라이트 크기에 맞춰 조정됨
  - Box Collider가 마음에 안 들면 다른 종류의 콜라이더를 대신 추가해도 됨(Box Collider를 넣으면 충돌 판정이 눈에 보이는 것과 다를 수 있음)



구현 방법

- 장애물과 포탈 둘 다 플레이어가 뚫고 지나갈 수 있는 것이 자연스럽다
  - 장애물이나 포탈에 닿았을 때 플레이어가 튕겨져 나오는 건 부자연스러움
  - Collider의 Is Trigger 설정을 체크하면 됨
- Is Trigger인 오브젝트와의 충돌 판정: OnTriggerEnter2D(Collider2D other), OnTriggerExit2D(Collider2D other)
  - 충돌한 두 오브젝트 중 적어도 하나가 Trigger인 경우 OnCollisionEnter2D, OnCollisionExit2D 대신 호출됨
  - Collision2D가 아니라 Collider2D 컴포넌트가 인수로 주어짐에 주의

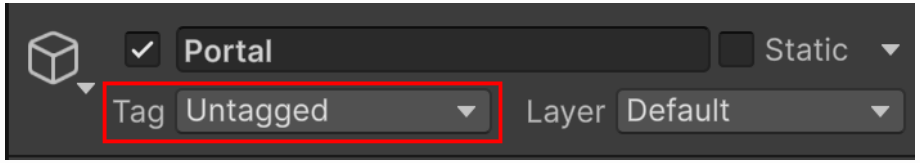
문제점

- 벽과 장애물/포탈 간에는 Collision이냐, Trigger냐로 구분되니 문제 없음
- 같은 Trigger 이벤트 안에서는 어떻게 구분할까?
- 지난 시간의 비슷한 문제: 벽과 벽이 아닌 것을 어떻게 구분할까?

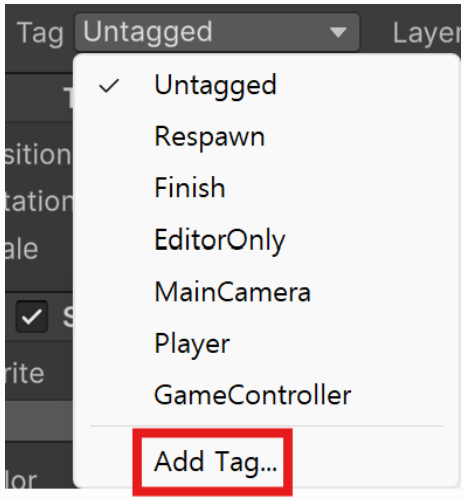
- OnTriggerEnter2D, OnTriggerExit2D에서는 Collider2D를 인수로 받음
  - 모든 컴포넌트는 .gameObject로 그것이 속한 게임오브젝트에 대한 정보를 알 수 있음(Collider2D other → other.gameObject)
  - 지금 충돌한 오브젝트가 어떤 오브젝트인지를 알면 장애물인지, 포탈인지 구분 가능
  - '어떤 오브젝트인지'는 어떻게 알까?

태그

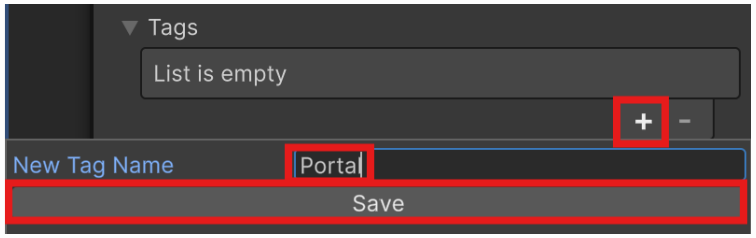
- 태그: 오브젝트의 종류를 구분하기 위해 Unity에서 제공하는 기능



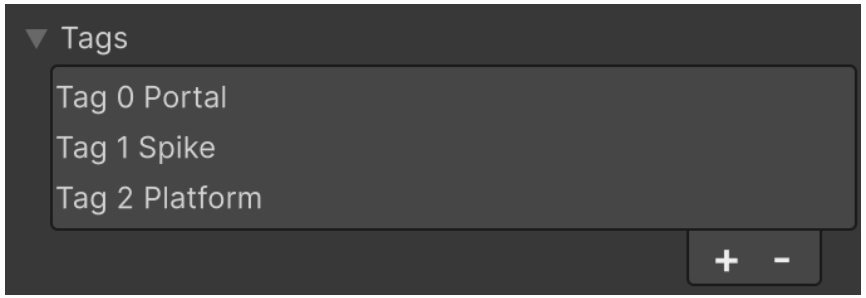
- 태그 추가하기



태그 클릭-Add Tag



\+ 클릭
New Tag Name에 태그 이름 입력
Save 클릭



Portal, Spike, Platform 태그 추가(순서는 무관)

이후 각 오브젝트에서 태그 지정

태그

- 스크립트에서 게임오브젝트의 태그에 접근하는 방법: **gameObject.tag**
- **Boolean CompareTag(string tag)**: 게임오브젝트의 태그가 tag와 일치하는지 여부를 반환
  - tag가 미리 등록된 태그 중에 없는 경우 오류
- **static GameObject FindWithTag(string tag)**: tag와 같은 태그를 가진 오브젝트 반환
  - 같은 태그인 오브젝트가 여러 개인 경우 그 중 첫 번째만 반환
  - 일치하는 태그가 없으면 **null** 반환

여담: static

- CompareTag를 호출할 때는 gameObject.CompareTag (자기 자신의 태그 비교) 또는 other.CompareTag (other라는 오브젝트의 태그 비교) 처럼 호출하면 됨
  - 즉, GameObject 클래스의 '인스턴스'인 객체들이 가진 CompareTag 메서드를 호출한다는 뜻

- 그런데 FindWithTag의 경우 반드시 GameObject.FindWithTag와 같이 호출해야 함
  - 여기서 대문자 GameObject는 GameObject 클래스 자체를 지칭

- 이유: FindWithTag는 GameObject 클래스에서 정의된 static 메서드이기 때문

여담: static

- 일반적인 속성/메서드: 객체별로 가지고 있음, 어떤 객체인지에 따라 달라질 수 있음
- static 속성/메서드: 객체가 아닌 클래스가 가지고 있음, 클래스 이름으로 호출됨

- static을 왜 쓸까? → 개별 객체의 상태와 무관하게 클래스의 모든 인스턴스가 공유하는 속성/함수가 필요할 때 사용
  - GameObject에서 객체를 찾거나 생성하는 등의 기능은 어떤 특정 객체와 관련된 것이 아니라 클래스 자체의 기능이므로 static을 사용

여담 2: CompareTag

- 그냥 gameObject.tag == tag처럼 써도 되는데 왜 CompareTag를 사용할까?
- 답: CompareTag가 더 효율적임

- 진짜 여담: 문자열 비교와 관련해서...
  - C#과 매우 비슷한 언어인 Java에서는 문자열 비교를 a == b가 아니라 a.equals(b)처럼 해야 함
  - 이유: Java에서는 a == b로 쓰면 두 문자열의 값이 같은지 비교하는 게 아니라 두 문자열의 참조(포인터)가 같은지 비교하기 때문
  - 다행히도 C#에서는 a == b로 쓰든 a.Equals(b)로 쓰든 상관 없음
  - 단, C#에서 문자열이 아니라 다른 클래스에 속하는 객체끼리 비교할 때는 Java와 동일함

실습 1: 가시와 포탈 구현하기

- 빈 칸에 들어갈 코드를 채워 보자

```
void OnTriggerEnter2D(Collider2D other)
{
    
}
```



- 힌트
  - 가시와 포탈의 구체적인 처리는 뒤에서 할 예정이므로, 지금은 경우에 따라 Debug.Log로 콘솔 출력만 하게 하거나 비워 두자
  - Debug.Log 사용법: Debug.Log("...");라고 쓰면 Unity 콘솔에 문자열이 출력됨

2. 프리팹

객체를 여러 개 생성하기

- 발판이나 가시 등은 한 맵에 여러 개가 필요할 수도 있음
- 그냥 복사+붙여넣기로 복제하면 되지 않나?
  - 문제점: 나중에 발판이나 가시를 수정해야 할 때, 복제된 모든 오브젝트를 일일이 수정해야 함

- DRY(Do not Repeat Yourself): 프로그래밍의 유명한 격언
  - 코드를 반복할 때의 문제점: 나중에 코드를 고쳐야 할 때 모든 부분을 찾아서 다 고쳐야 함
  - 유지보수성을 위해서 단순 반복은 최소화하는 것이 옳음

프리팹

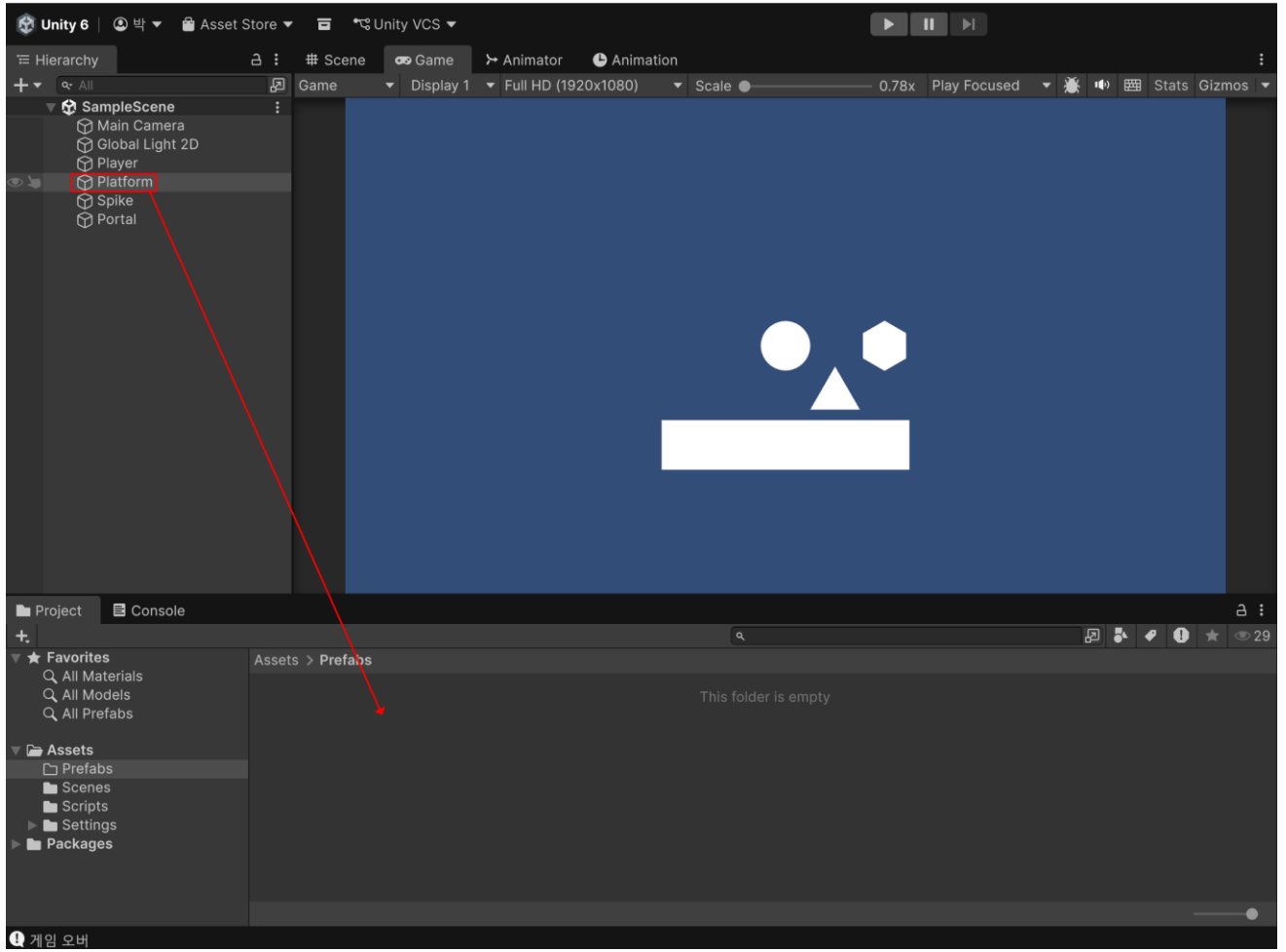
- 오브젝트에 대한 청사진
- 클래스 : 객체 = 프리팹 : 게임오브젝트

- 프리팹은 에셋으로 저장 가능
- 저장된 프리팹을 원본으로 하여 여러 게임오브젝트를 생성 가능
- 프리팹을 바꾸면 프리팹으로부터 생성된 게임오브젝트들에도 변경사항이 적용됨
- 프리팹으로 생성된 게임오브젝트라 하더라도 프리팹과 다른 부분을 만들고 싶으면 오버라이드할 수 있음

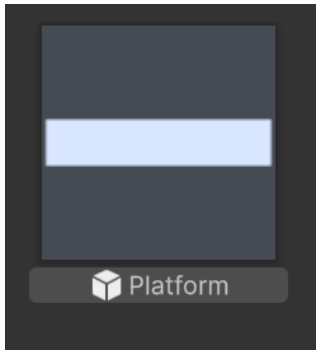
프리팹



Prefabs
폴더 생성



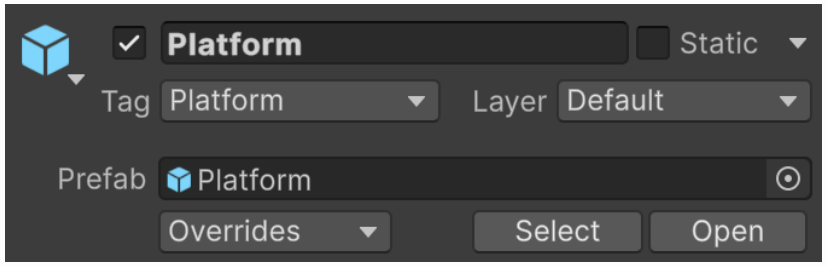
오브젝트 선택-Prefabs 폴더로 드래그&드롭



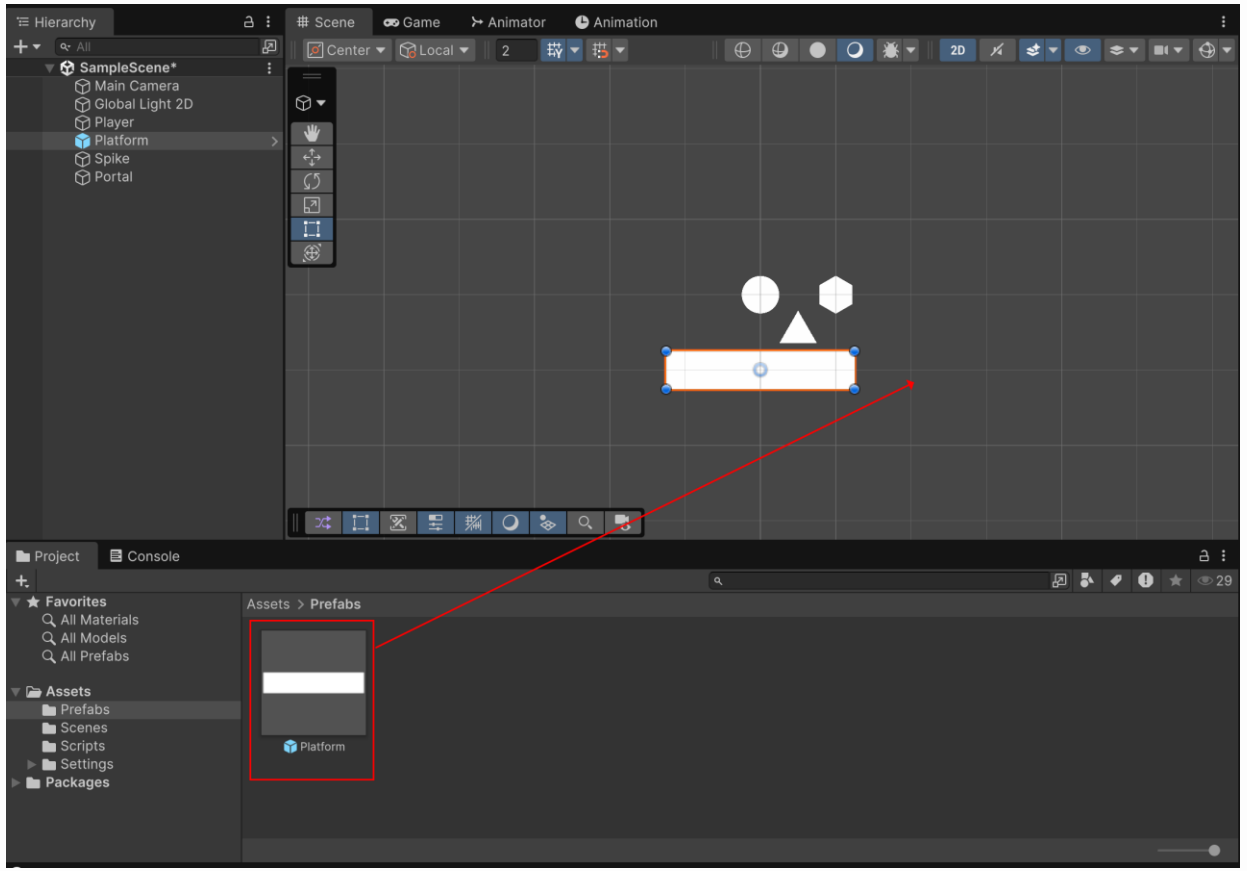
프리팹이
생성됨

Player,
Platform,
Spike,
Portal
모두에
대해서 수행

프리팹



프리팹으로부터 생성된
오브젝트에는 원본 프리팹이
나타남



프리팹을 씬 뷰 또는 하이어라키 뷰에
드래그&드롭하면 프리팹을 원본으로 하는 새
게임오브젝트가 생성됨

프리팹

- 하지만 프리팹으로 할 수 있는 일은 이 뿐만이 아닌데...
- 프리팹은 '에셋'이라고 했음
- 에셋은 컴포넌트에 속성으로 넣을 수 있음
- 프리팹을 컴포넌트의 속성으로 받아서 뭘 할 수 있을까?

3. 스테이지 동적 생성

동적 생성

- 가시에 닿았을 때, 포탈에 닿았을 때 처리를 디버그 출력으로 대신한 이유: 가시에 닿으면 '처음 상태로 복구'해야 하고, 포탈에 닿으면 '다음 스테이지로 이동'해야 하는데, 두 경우 모두 '스테이지'라는 개념이 필요함
- 현재 구현 방식에서는 맵이 고정되어 있음

- 여러 스테이지를 만들고, 원하는 스테이지를 불러올 수 있으려면 뭐가 필요할까?

- 일단 오브젝트가 처음부터 씬에 배치되어 있으면 안 된다
  - 어떤 스테이지인지에 따라 배치되는 오브젝트가 다르기 때문
- 처음부터 오브젝트가 배치되는 게 아니라면, 지금이 몇 번째 스테이지인지를 보고 오브젝트를 동적으로 배치해야 한다
- 오브젝트를 어떻게 동적으로 배치할 수 있을까?

동적 생성

- GameObject.Instantiate
  - 게임오브젝트를 생성하는 static 메서드
  - 호출 형태
    - static GameObject Instantiate(Object original): original을 원본으로 하는 게임오브젝트 생성
    - static GameObject Instantiate(Object original, Transform parent): 동일하지만 parent의 Transform을 생성된 오브젝트의 부모로 설정
    - static GameObject Instantiate(Object original, Vector3 position, Quaternion rotation): 동일하지만 위치를 position으로 설정, 회전을 rotation으로 설정
  - 같은 함수이지만 인수 값이 다름 → 메서드 오버로딩

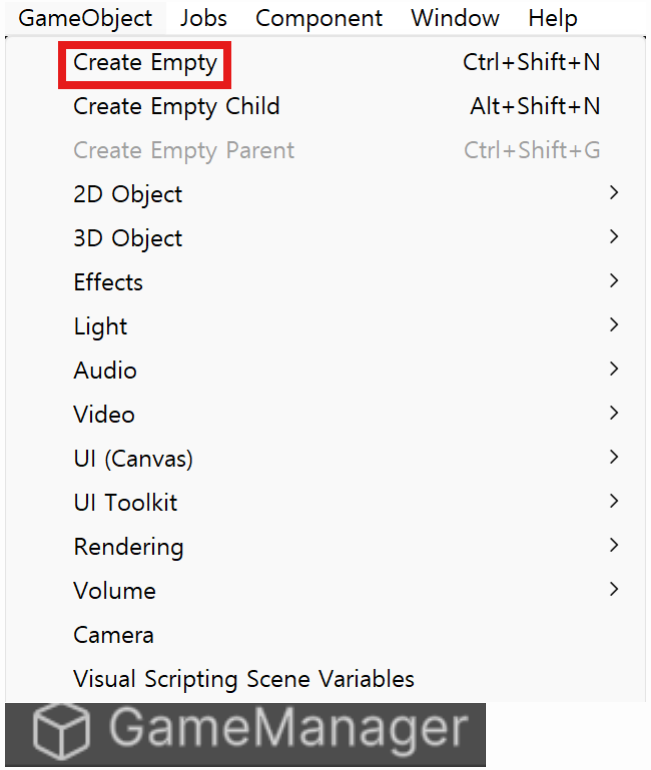
동적 생성

- static void Destroy(Object obj)
  - obj로 지정된 게임오브젝트를 삭제
  - 게임오브젝트 뿐만 아니라 컴포넌트도 삭제할 수 있음
  - ex) Destroy(this): 이 스크립트를 스크립트가 붙어 있는 게임오브젝트에서 삭제

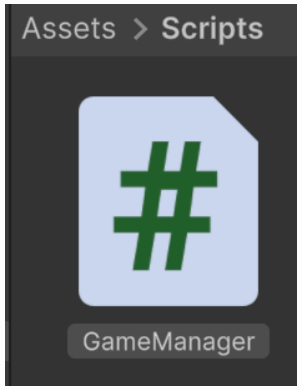
동적 생성

- 구현 방식
  - 모든 오브젝트들의 생성/소멸을 관리하는 게임오브젝트를 만든다
  - 이 게임오브젝트에서 스테이지 시작 시점에 Instantiate를 통해 맵에 들어가는 모든 게임오브젝트들을 생성하고, 스테이지가 끝나면 생성했던 게임오브젝트들을 모두 삭제

동적 생성 구현



GameObject-Create Empty로 GameManager 생성



Assets/Scripts에 GameManager 스크립트 추가 후 GameManager 오브젝트에 적용

동적 생성 구현

- 코드



코드를 작성하기는 아직 이르다

- 답해야 할 두 가지 질문
  - Instantiate는 원본 오브젝트를 필요로 하는데, 그 원본 오브젝트를 어떻게 받아올 것인가?
  - 스테이지의 정보는 어떤 식으로 저장할 것인가?

- 질문에 대한 대답
  - 원본 오브젝트는 프리팹을 SerializeField로 받아온다
  - 스테이지 정보는 ScriptableObject로 저장한다

4. SerializeField

SerializeField가 뭘까?

- 이전부터 종종 언급됨

- 스크립트를 게임오브젝트에 붙이면 스크립트는 컴포넌트가 된다고 했음
- 우리가 지금까지 봐온 컴포넌트들은 값을 입력받을 수 있는 필드 같은 것들이 존재했음
  - Rigidbody의 Body Type, Collider의 Is Trigger 등

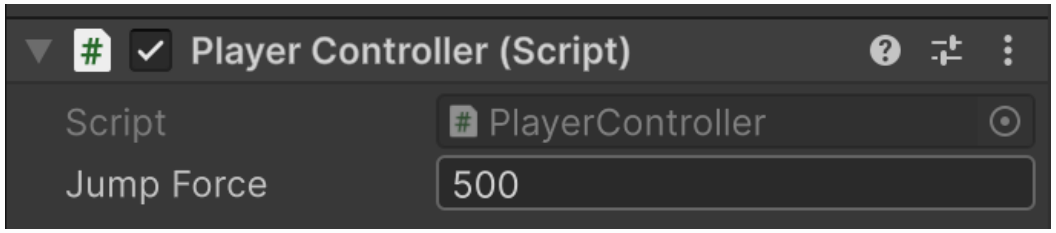
- 스크립트는 '직접 만드는 컴포넌트'라고 했는데, 그럼 스크립트에서도 이런 필드를 만들 방법이 없을까?
- → 이게 SerializeField가 하는 역할

SerializeField 만들기

- PlayerController에 다음과 같은 내용을 추가해보자

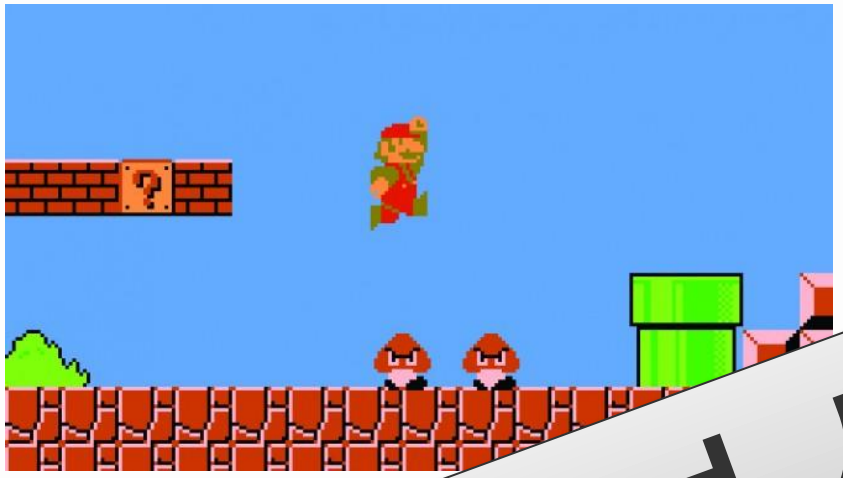
```
private float _speed = 5.0f;
[SerializeField] private float _jumpForce = 500.0f;
private Rigidbody2D _rb;
private bool _isJumping = false;
```

- 결과



- 에디터에서 Jump Force 값을 500이 아닌 다른 값으로 바꾸고 플레이해 보면 점프력이 달라진다

단적인 예시



- 캐릭터가 더 높이 점프하게 만
- 나쁜 명령: 캐
  - 캐릭터 점프력을 jumpForce에서 지정하고 있지? 그 부분만 따로 테스트해볼 수 있게 serializeField로 바꿔줘.
  - 사실 프로그래밍을 할 줄 알면 이 정도는 AI한테 명령할 것도 없음

복선 회수

것보다는 덜
점프하게 만들어줘'라고

법을 쓸 줄 아는 것도 아니고...

SerializeField 만들기

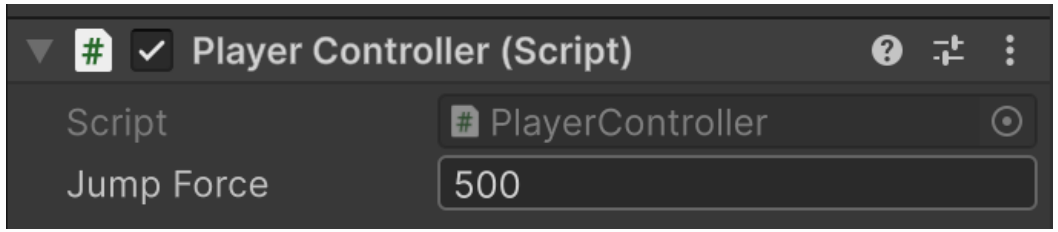
- 이 방법의 장점
  - 코드상에서 상수 값을 바꿔가면서 테스트하려면 컴파일하는 데 시간이 걸리지만, 에디터상에서 값을 바꾸면 다시 컴파일할 필요가 없음
  - 더 보편적으로 말하면, 코드상으로는 private이므로 다른 코드가 값을 수정할 위험은 없지만, 에디터에서는 값을 바꿀 수 있음 → 객체 지향의 원칙에 부합
  - 프로그래밍을 모르는 사람도 에디터만 다룰 줄 알면 설정값 변경 가능 → 기획과의 협업에 유리하다

참고: Unity에서 public 변수에 대한 직렬화

- 사실 클래스에서 변수를 private이 아니라 public으로 선언해도 스크립트 컴포넌트에서 필드로 입력을 받을 수는 있음

```
public float _jumpForce = 500.0f;
```

- 즉 이렇게 해도 아래처럼 보임



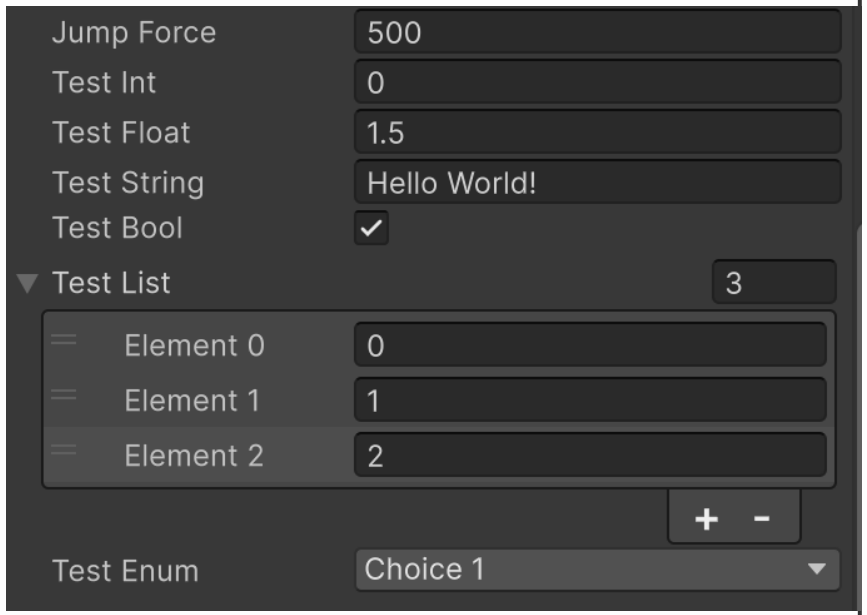
- 그럼에도 불구하고 SerializeField를 쓰는 이유는 변수를 private으로 설정하면서도 에디터에 노출시킬 수 있기 때문
  - public을 함부로 쓰면 한 코드에서 실수로 다른 코드의 값을 변경해버릴 위험이 있음

SerializeField

- 말 그대로 '직렬화' 가능한 '필드'라는 뜻의 어노테이션
  - 직렬화: 객체의 데이터를 저장 가능한 형태로 변형함
  - python의 pickle이나 JSON 같은 것들의 역할

- 어떤 값들이 어떻게 직렬화될까?
  - 정수(int 등), 실수(float, double 등): 숫자 입력 필드
  - 문자열(string): 문자열 입력 필드
  - 불린(bool): 체크박스
  - 리스트(List<T>): 리스트 입력
  - 열거형(enum): 드롭다운 박스

- 제시된 자료형 외에 딕셔너리처럼, 명시적으로 직렬화 가능하다고 선언되지 않은 자료형들은 직렬화되지 않음에 주의



참고: 열거형

- 상수(정수)에 이름을 붙여 관리하는 기능
- 필요한 이유: 프로그래밍에서는 인수나 반환값으로 간단한 정수 형태의 값을 주고받는 경우가 많은데, 이때 정수 값의 의미를 직관적으로 파악하기 어려움
- 열거형을 사용하면 정수에 유의미한 이름을 붙여 관리하니 코드의 의미 파악이 쉬워짐
- 비유: 404 오류라고 하면 무슨 오류인지 모르지만, 404 Not Found라고 하면 찾는 주소가 없다는 뜻이라는 것을 알기 쉬움

- 문법

```
public enum TestEnum{
    Choice1,
    Choice2,
    Choice3
}
```

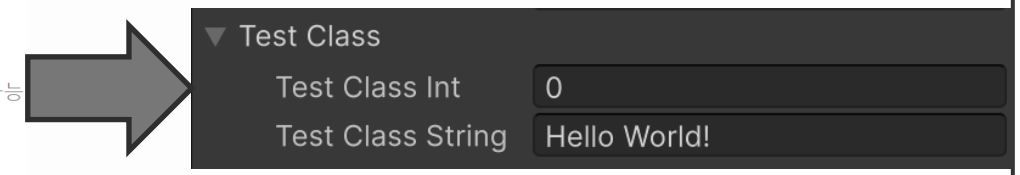
```
public static void Main(string[] args) {
    TestEnum testEnum = TestEnum.Choice1;
    switch(testEnum){
        case TestEnum.Choice1:
            Console.WriteLine("choice 1");
            break;
    }
}
```

SerializeField

- 여러 필드를 하나로 묶어서 직렬화하는 방법
  - 필드들을 하나의 클래스 안에 정의하고, 해당 클래스에 [Serializable] 어노테이션 추가
  - Serializable 어노테이션은 System 네임스페이스에 있으므로 using System; 필요

```
[Serializable]
2 사용 위치
public class TestClass
{
    [SerializeField] private int _testClassInt = 0;  직렬화 가능
    [SerializeField] private string _testClassString = "Hello World!";  직렬화 가능
}

[SerializeField] private TestClass _testClass = new TestClass();  직렬화 가능
```



SerializeField

- 그리고...
  - GameObject를 비롯한 에셋이나 컴포넌트들도 SerializeField로 받을 수 있음
  - 프리팹은 타입이 GameObject인 에셋으로 취급됨

- ⇒ SerializeField로 플레이어/발판/장애물/포탈에 해당하는 프리팹을 받아서 Instantiate의 인수로 넘겨주면 됨

SerializeField로 프리팹 받기

- 플레이어 프리팹, 발판 프리팹, ...을 하나하나 별개의 SerializeField로 받을 수도 있지만, 이 경우 새 프리팹이 생기면 변수도 하나 더 추가해야 해서 불편함
- List<GameObject>를 SerializeField로 할 수도 있으나, 인덱스를 프리팹에 대응시키면 몇 번 인덱스가 어느 프리팹인지 알기 어려움
- EntityType이라는 열거형을 정의해 두고 Dictionary<EntityType, GameObject>처럼 딕셔너리를 쓸 수도 있겠지만 딕셔너리는 직렬화되지 않음

- 해법: EntityType과 GameObject를 속성으로 가지는 클래스 PrefabDictEntry를 정의해서, List<PrefabDictEntry>를 SerializeField로 받고 시작 시점에 이를 Dictionary<EntityType, GameObject>로 파싱

SerializeField로 프리팹 받기

```
public enum EntityType
{
    Player,
    Platform,
    Spike,
    Portal,
    None
}

[Serializable]
3 사용 위치
public class EntityDictEntry
{
    public EntityType EntityType;  직렬화 가능
    public GameObject GameObject;  직렬화 가능

    public EntityDictEntry(EntityType entityType, GameObject gameObject)
    {
        EntityType = entityType;
        GameObject = gameObject;
    }
}
```

실습 2: 빈 부분 채우기

- 앞의 코드 바로 다음에 이어서, EntityDictEntry들의 리스트를 SerializeField로 입력받아 _entityDict를 채우는 코드를 만들어보자

- 정답

```
[redacted]                                                    직렬화 가능
private Dictionary<EntityType, GameObject> _entityDict = new Dictionary<EntityType, GameObject>();

// Start is called once before the first execution of Update after the MonoBehaviour is created
이벤트 함수
void Start()
{
    [redacted]
}
```

4. ScriptableObject

ScriptableObject

- SerializeField가 '스크립트를 컴포넌트처럼 만들어주는' 역할이었다면, ScriptableObject는 '스크립트를 에셋처럼 만들어주는' 역할
  - 실제로 프로젝트 탭에서 우클릭으로 생성할 수도 있음

- 씬과 무관하게 존재하는 데이터를 저장하는 데 사용
  - 캐릭터 스탯 정보, 맵 정보 등

ScriptableObject 만들기



Assets/Scripts에
MapData 추가

```
using System;
using System.Collections.Generic;
using UnityEngine;

[CreateAssetMenu(menuName = "Scripts/Map Data")]
애셋 사용 위치가 없습니다
public class MapData : ScriptableObject
{
    [Serializable]
    2 사용 위치
    public class MapDataRow
    {
        public List<GameManager.EntityType> entityTypes = new List<GameManager.EntityType>();
    }

    public List<MapDataRow> mapDataRows = new List<MapDataRow>();   직렬화 가능
}
```

MapData에 다음과 같이 입력

코드 분석

```
using System;
using System.Collections.Generic;
using UnityEngine;

[CreateAssetMenu(menuName = "Scripts/Map Data")]
애셋 사용 위치가 없습니다
public class MapData : ScriptableObject
{
    [Serializable]
    2 사용 위치
    public class MapDataRow
    {
        public List<GameManager.EntityType> entityTypes = new List<GameManager.EntityType>();
    }

    public List<MapDataRow> mapDataRows = new List<MapDataRow>();
}
```

프로젝트 뷰를 우클릭하면 나오는 메뉴에서 Scripts/Map Data를 선택하면 이 ScriptableObject를 에셋으로 만들 수 있게 설정

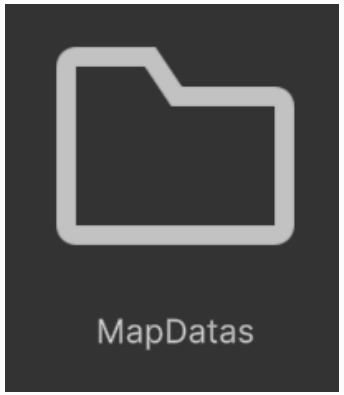
ScriptableObject를 상속

중첩 리스트는 Serializable하지 않으므로 안쪽 리스트를 Serializable한 별개의 클래스로 선언

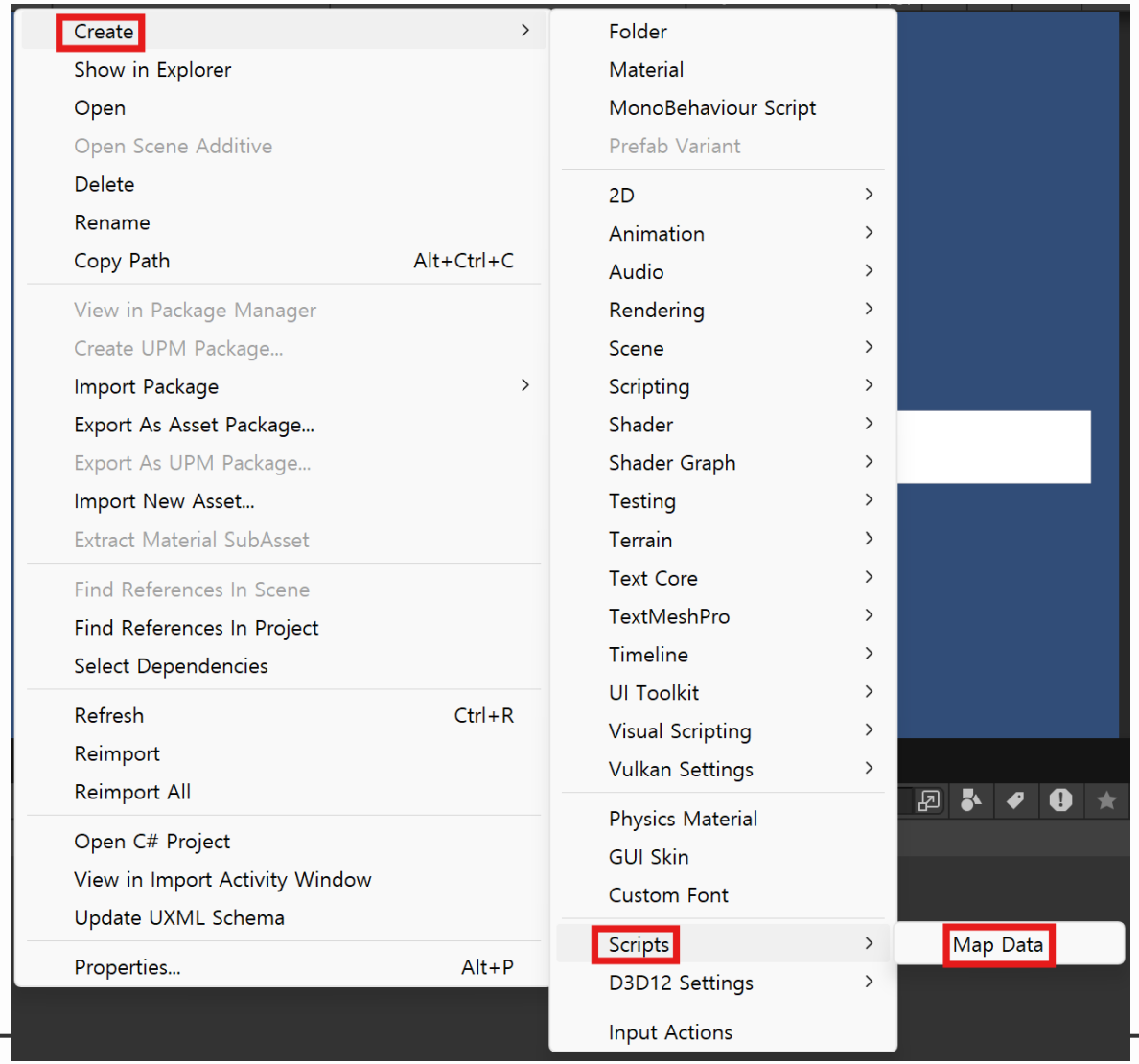
List<EntityType>을 속성으로 가짐 public이므로 SerializeField로 선언할 필요 없음

ScriptableObject에서 편집하고 싶은 모든 속성은 직렬화 가능해야 함 여기서는 public이므로 SerializeField로 선언하지 않는 것

ScriptableObject 만들기



Assets에
MapDatas
폴더 추가

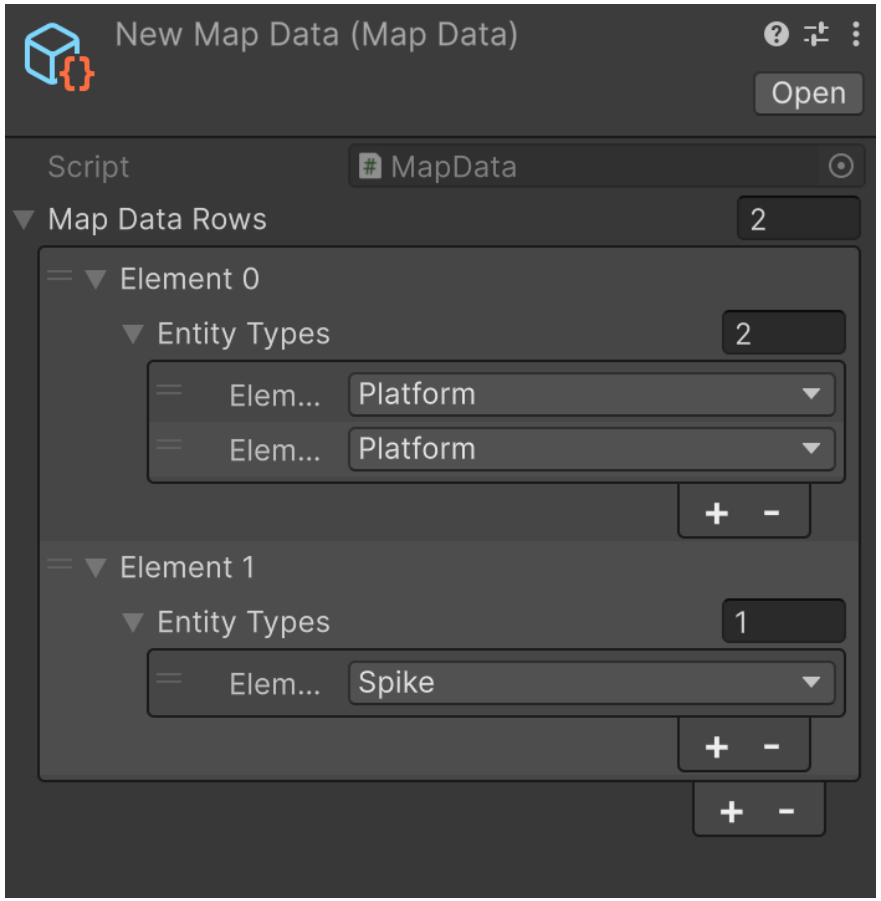


MapDatas 폴더 안에
들어와서 우클릭-Create-
Scripts-Map Data 선택



MapData
에셋이
생성됨

ScriptableObject 만들기



인스펙터 창에서 맵 정보를 편집할 수 있음

여담: MapData의 구조에 관한 고찰

- 지금 당장은 '같은 타입의 오브젝트라면 모두 똑같이 행동한다'는 보장이 있으므로 EntityType만 MapData에 저장해도 무방
- 만약 같은 타입의 오브젝트더라도 서로 다르게 행동할 수 있다면?
  - ex) 일정 주기로 화살을 발사하는 화살 발사대가 있는데, 화살의 주기를 마음대로 정할 수 있음
- 이런 경우 EntityType만 저장하는 것이 아니라 더 구체적인 정보를 저장할 필요가 있음
- 한 가지 해결 방법: Entity라는 클래스를 만들어서 EntityType 대신 Entity를 리스트로 저장(당연히 Entity도 ScriptableObject여야 함)
  - 서로 다른 오브젝트에 대한 데이터는 Entity를 상속하게 만듦
  - ex) 화살 발사대 오브젝트의 경우 Entity가 가지고 있는 정보 + 화살 발사 주기 데이터까지 속성으로 갖도록 함
  - 이 경우 EntityType은 어느 오브젝트인지를 판별하기 위한 목적으로 Entity의 속성 중 하나로 넣을 수 있음

- 이런 고민 없이 코드를 짜면 나중에 크게 수정할 일이 생긴다!(경험담)

실습 3: 종합

- 맵 데이터를 ScriptableObject로 저장하는 것까지 완료
- 그런데 이 맵 데이터를 GameManager에서 접근할 수 있어야 한다
- GameManager를 수정해서 맵 데이터들을 에디터에서 입력받을 수 있게 하자

- 힌트
  - 변수 하나만 추가하면 된다
  - ScriptableObject도 결국은 클래스다!

실습 3: 종합

- 정답

```
[SerializeField] private List<MapData> _mapDatas = new List<MapData>();
```

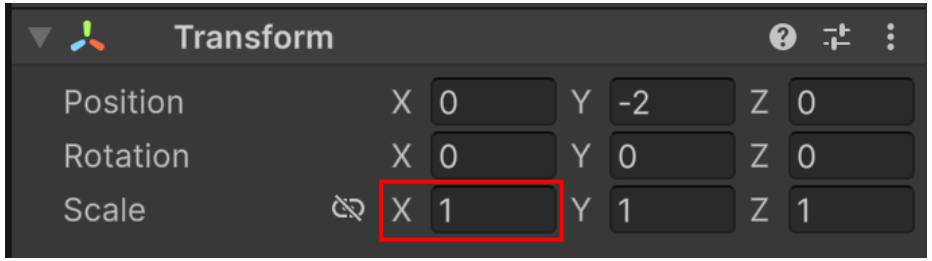
5. 동적 생성 완성하기

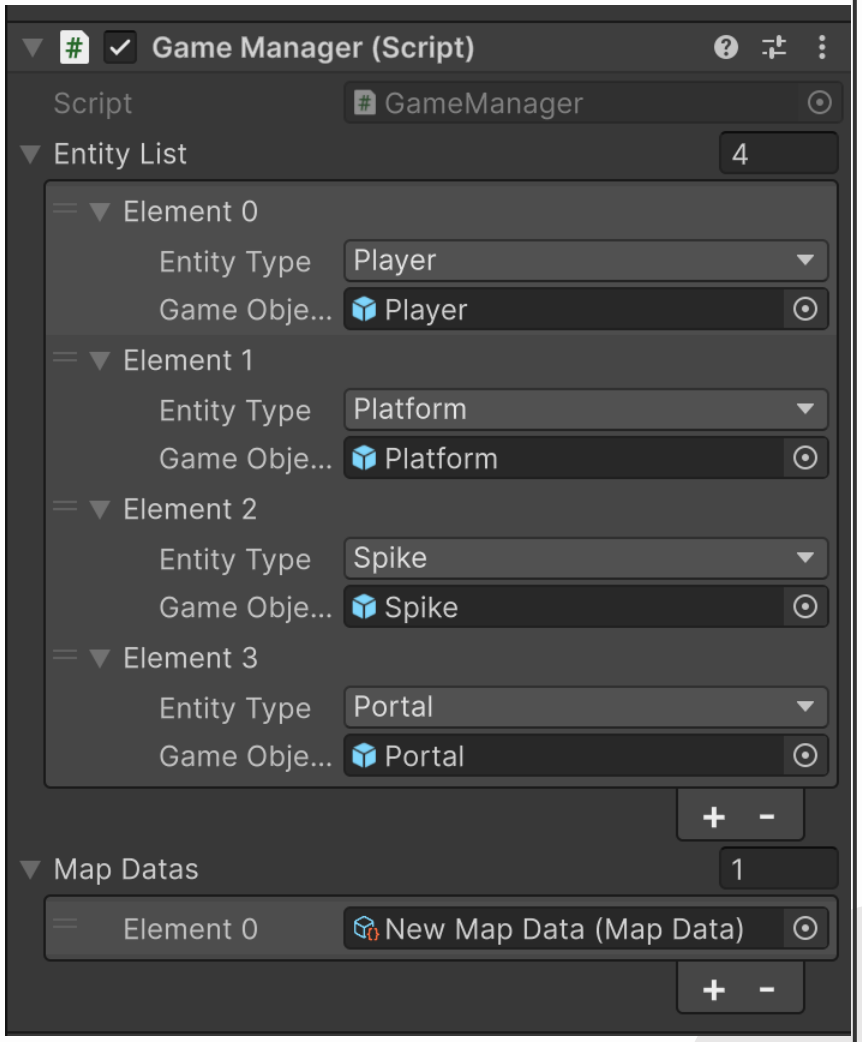
동적 생성 완성하기

- 지금까지 한 일
  - SerializeField로 프리팹 및 맵 데이터 받기
  - ScriptableObject로 맵 데이터 저장하기
- 이제 해야 할 일
  - GameManager에서 맵 데이터에 맞춰서 게임오브젝트들을 Instantiate하게 만들기

동적 생성 완성하기

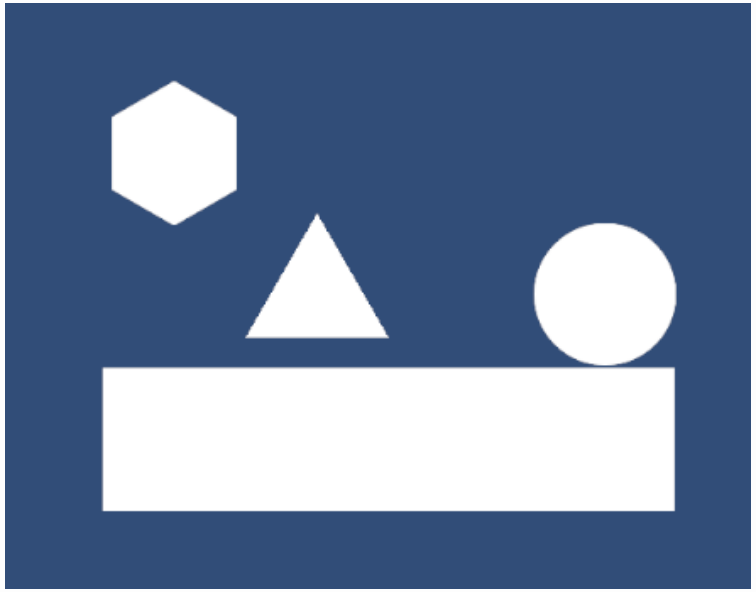
- 사전 준비
  - GameManager의 Game Manager 스크립트에서 Entity List 채우기(None은 제외)
  - GameManager의 Game Manager 스크립트에서 Map Datas에 Map Data 추가하기
  - 씬에서 GameManager를 제외한 우리가 생성한 모든 게임오브젝트 삭제하기
  - Platform 프리팹의 Transform에서 Scale의 x값을 1로 되돌리기

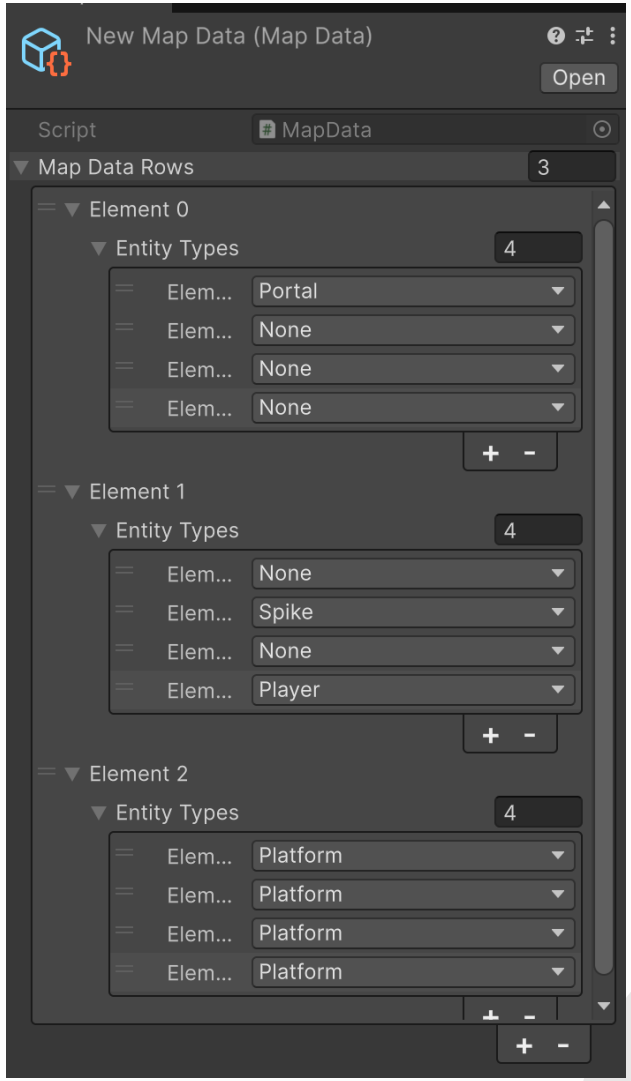




동적 생성 완성하기

- 테스트용 맵 데이터
- 생성된 모습





실습 4: 퍼즐 조각 맞추기

- 빈 칸에 들어갈 내용을 채워 보자
- 힌트는 다음 장 참조

```
private void LoadMap(int idx)
{
    int height = _mapDatas[   ].            .    ;
    for (int i = 0; i < height; i++)
    {
        int width = _mapDatas[   ].                .           .     ;
        for (int j = 0; j < width; j++)
        {
            Vector3 coord = IndexToCoord(                                  );
            EntityType entityType = _mapDatas[idx].mapDataRows[i].entityTypes[j];
            if (entityType != EntityType.None)
            {
                Instantiate(          [          ], coord, Quaternion.identity);
            }
        }
    }
}

1 사용 위치
private Vector3 IndexToCoord(int rowIdx, int colIdx, int width, int height)
{
    return new Vector3(colIdx - width / 2.0f, height / 2.0f - rowIdx, 0);
}
```

실습 4: 퍼즐 조각 맞추기

- 힌트
  - IndexToCoord는 오브젝트들을 화면 정중앙에 배치할 수 있도록, 행과 열을 입력하면 알맞은 위치를 반환하는 헬퍼 함수이다
  - MapData 오브젝트의 구조를 다시 잘 살펴보자
  - 리스트의 길이: list.Count
  - i는 행(row), j는 열(column)
  - MapDataRow는 세로줄을 나타낸다

- 테스트 코드

```
void Start()
{
    foreach(EntityDictEntry entityDictEntry in _entityList)
    {
        _entityDict[entityDictEntry.EntityType] = entityDictEntry.GameObject;
    }

    LoadMap( idx: 0);
}
```

실습 4: 퍼즐 조각 맞추기

- 정답

```
private void LoadMap(int idx)
{
    int height = _mapDatas[idx].mapDataRows.Count;
    for (int i = 0; i < height; i++)
    {
        int width = _mapDatas[idx].mapDataRows[i].entityTypes.Count;
        for (int j = 0; j < width; j++)
        {
            Vector3 coord = IndexToCoord( rowIdx: i,  colIdx: j, width, height);
            EntityType entityType = _mapDatas[idx].mapDataRows[i].entityTypes[j];
            if (entityType != EntityType.None)
            {
                Instantiate(_entityDict[entityType], coord, Quaternion.identity);
            }
        }
    }
}
```

실습 5: 끝난다는 것은 다시 시작된다는 것을

- 오브젝트를 생성한 것까지는 좋았는데, 오브젝트를 소멸시킬 필요도 있음
- 한 스테이지가 끝나고 다음 스테이지가 시작될 때, 생성된 모든 오브젝트를 삭제하고 다시 시작해야 하기 때문

- LoadMap으로 생성된 모든 오브젝트를 소멸시키는 RemoveLoadedMap 함수를 만들어보자

- 힌트
  - RemoveLoadedMap은 반환값도, 인수도 없으며, private 함수이다
  - LoadMap으로 생성된 오브젝트가 무엇인지를 어떤 식으로 저장하고 있다가, RemoveLoadedMap에서는 이 저장된 오브젝트들만 일일이 Destroy하면 되지 않을까?
  - LoadMap도 약간 수정해야 하는데, Instantiate가 게임오브젝트를 반환한다는 사실을 잊지 말자

실습 5: 끝난다는 것은 다시 시작된다는 것을

- 정답

```
private List<GameObject> _spawnedObjects = new List<GameObject>();

3 사용 위치
private void LoadMap(int idx)
{
    int height = _mapDatas[idx].mapDataRows.Count;
    for (int i = 0; i < height; i++)
    {
        int width = _mapDatas[idx].mapDataRows[i].entityTypes.Count;
        for (int j = 0; j < width; j++)
        {
            Vector3 coord = IndexToCoord( rowIdx: i,  colIdx: j, width, height);
            EntityType entityType = _mapDatas[idx].mapDataRows[i].entityTypes[j];
            if (entityType != EntityType.None)
            {
                _spawnedObjects.Add(Instantiate(_entityDict[entityType], coord, Quaternion.identity));
            }
        }
    }
}
```

```
private void DestroyLoadedMap()
{
    foreach (GameObject go in _spawnedObjects)
    {
        Destroy(go);
    }
    _spawnedObjects.Clear();
}
```

기나긴 여정 끝에...

- 다시 가시에 닿았을 때와 포탈에 닿았을 때에 대한 처리를 해 보자

- 맵 로딩은 GameManager가 담당하므로, GameManager에서 public 메서드를 제공해서 두 경우에 호출할 수 있게 하는 것이 자연스러움

- PlayerController에서 GameManager의 메서드를 어떻게 호출하는가?
  - GameObject.Find(string name): name과 같은 이름의 게임오브젝트를 하나 반환
  - Find로 찾은 오브젝트에서 GetComponent<GameManager>()를 호출하여 스크립트를 얻은 뒤, 이 스크립트에서 메서드를 호출하면 됨

GameManager 구현

```
private int _curIdx = 0;

// Start is called once before the first execution of Update after the MonoBehaviour is created
이벤트 함수
void Start()
{
    foreach(EntityDictEntry entityDictEntry in _entityList)
    {
        _entityDict[entityDictEntry.EntityType] = entityDictEntry.GameObject;
    }

    LoadMap( idx: 0);
    _curIdx = 0;
}
```

GameManager 구현

```
public void ResetMap()
{
    DestroyLoadedMap();
    LoadMap(_curIdx);
}

1 사용 위치
public void ToNextMap()
{
    DestroyLoadedMap();
    _curIdx = (_curIdx + 1) % _mapDatas.Count;
    LoadMap(_curIdx);
}
```

PlayerController 구현

```
void Start()
{
    _rb = GetComponent<Rigidbody2D>();
    _gameManager = GameObject.Find("GameManager").GetComponent<GameManager>();
}
```

```
private GameManager _gameManager;
```

PlayerController 구현

```
void OnTriggerEnter2D(Collider2D other)
{
    if (_gameManager == null)
    {
        return;
    }

    if (other.CompareTag("Spike"))
    {
        _gameManager.ResetMap();
    }

    if (other.CompareTag("Portal"))
    {
        _gameManager.ToNextMap();
    }
}
```

Start에서 _gameManager를 찾아오니 _gameManager가 null이 아님을 보장할 필요가 없는 것 아닌가?

→ Start보다 OnTriggerEnter2D가 먼저 호출되는 경우가 있을 수 있음 (Start는 생각보다 매우 늦게 호출됨!)

이번 주차 요약

- 태그
- 동적 생성의 필요성
- SerializeField
- ScriptableObject

- 다음 주차: 중간고사 휴강 이후 10/21